CDT204 – Computer Architecture

> Date: Nov. 8th, 2024
>
> Time: 8:30 – 12:30
>
> Help: Small calculator is allowed in the exam

The exam has 56 points. The grades will be awarded as follows:

- 3: 28 Points
- 4: 42 Points
- 5: 50 Points

Important Notes:

- Give as full an answer as possible to obtain full marks. All calculations, approximations, assumptions and justifications must be reported for full credit unless stated otherwise. Please use figures and examples to clarify.
- If you do not understand a question clearly, you are allowed to call the teacher and ask.
- Write the question and part number on each page clearly.
- Answer each question on a separate sheet.
- In case you might have forgotten:

  1GB = 2^{10}MB = 2^{20}KB = 2^{30}B

  1 sec = 10^3 ms = 10^6 μs = 10^9 ns = 10^{12} ps

  $$S = \frac{T_0}{\frac{T_A}{F} + (T_0 - T_A)}$$

Live long and prosper

Task 1 (7p)

a) A program spends 75% of its time doing multiply instructions. If we can redesign the multiplier in the pipeline in a way that it gets 3x faster, how much faster does the program run on this new design? (**2p**)
b) What is the limit on the maximum speedup if the multiplier from part (a) is made infinitely fast? (**2p**)
c) For a 4-bit two's complement system, show the following operations in binary: (**3p**: a(1p), b(2p))
    a. 5-7
    b. 11-6

Task 2 (8p)

When a program is adapted to run on multiple processors in a multiprocessor system, the execution time on each processor is comprised of computing time and the overhead time required for locked critical sections and/or to send data from one processor to another.

Assume a program requires $t = 200s$ of execution time on one processor. When run on p processors, each processor requires t/p seconds, as well as an additional $5s$ of overhead, irrespective of the number of processors.

a) Compute the per-processor execution time for 2, 4, 8, 16, 32, 64, and 128 processors. For each case, list the corresponding speedup relative to a single processor and the ratio between actual speedup versus ideal speedup (speedup if there was no overhead). (**4p**)
b) Draw a chart with the number of processors on the x-axis and the speedup on the y-axis. Plot both actual speedup and ideal speedups that you calculated for the previous part into this graph. Discuss your findings. (**4p**)

Task 3 (14p)

Consider a hypothetical machine called SIC, for Single Instruction Computer. As its name implies, SIC has only one instruction: *subtract and branch if negative*, or sbn for short. The sbn instruction has three operands, each consisting of the address of a word in memory:

```
sbn a, b, c     #Mem[a] = Mem[a] – Mem[b]; if (Mem[a]<0) go to c
```

The instruction will subtract the number in memory location b from the number in location a, and place the result back in a, overwriting the previous value. If the result is greater than or equal to 0, the computer will take its instruction just after the current instruction. If the result is less than 0, the next instruction is taken from memory location c. SIC has no registers and no instructions other than sbn.

A. Identify what the following code does and write the equivalent code in C (The ".+1" in the code means "The address after this one") (**3p**).

```
start: sbn temp, temp, .+1
       sbn temp, a, .+1
       sbn b, b, .+1
       sbn b, temp, .+1
```

B. Assume that Mem[0] has a constant value of zero and Mem[1] has a constant value of one (they both cannot be written to). Also assume that memory locations 2 to 8 can be used for storing temporary values (main program and its data will not be stored there). How could the following MIPS instructions be done in SIC code? (**11p**: a(3p), b(4p), c(4p))
   a. J
   b. ADD
   c. MOVE

Task 4 (6p)

Consider two different implementations M1 and M2 of the same instruction set. There are four classes of instructions (A, B, C, and D) in the instruction set. M1 has a clock rate of 3.5 GHz and M2 has a clock rate of 4.5 GHz. For a given program, the fraction of all instructions that belong to a particular class, and the average number of cycles for each instruction in the two implementations are as below:

Class	Percentage of instructions	CPI on M1	CPI on M2
A	10%	1	2
B	35%	2	2
C	25%	3	4
D	30%	4	4

a) Which implementation of the instruction set can run the program faster? By how much? (**4p**)
b) What should be the clock speed of the slower implementation so that it can have the same speed as the faster implementation? (**2p**)

Task 5 (15p)

We are trying to reverse-engineer the characteristics of a cache system. To do so, we have come up with three patterns that access various bytes in the system in an attempt to determine the following cache characteristics (the numbers in parentheses represent possible options, we know that it can be only one of these values):

- Cache Block Size (8, 16, 32, 64 or 128 B)
- Cache Associativity (1-, 2-, 4- or 8-way)
- Cache Size (4 or 8 KB)

The only statistic that we can collect on this system is *cache hit rate* after performing the access patterns. This is what we observe:

Sequence	Addresses Accessed (Oldest -> Youngest)								Hit Rate
1	0	4	8	16	64	128			1/2
2	31	8192	63	16384	4096	8192	64	16384	5/8
3	32768	0	129	1024	3072	8192			1/3

Assume that the cache is initially empty at the beginning of the first sequence, but not at the beginning of the second and third sequences. The sequences are executed back-to-back, i.e., no other access takes place between the three sequences. Thus, at the beginning of the second/third sequence, the contents are the same as at the end of the first/second sequence. Based on what you observe, what are the following characteristics of the cache?

a) Cache Block Size (**5p**)
b) Cache Associativity (**5p**)
c) Cache Size (**5p**)

TASK 6 (p)

Parallelism can be achieved at different levels. Namely: bit level, instruction level, task level and data level.

a) "*Multiple issuing*" tries to achieve parallelism at which level? How does it achieve that? Explain it with examples (**2p**)

b) Cache coherence can be a problem in designing shared memory multi-processors. Explain the problem and suggest solutions with examples. (**2p**)

c) Although GPGPUs have more processing units than a multicore CPU, they normally have caches which are much smaller. Why? (**2p**)